CSE232 시스템프로그래밍(캡스톤디자인)

7주차 2차시 · V03

프로세스 심화: 측정하고 실험하기

경민기

KOREATECH
한국기술교육대학교

오늘의 지도

1부 메모리	2부 상태와 CPU	3부 수명	4부 한계
요구 페이징을 센다 pagetouch minflt · VmRSS · PSS	/proc/PID/stat 해석 pstat · cpu_wait 틱 · %CPU · 문맥 교환	좀비와 고아를 만든다 reap_show Z · <defunct> · PPID 1	일부러 넘겨 본다 limit_lab EMFILE · ENOMEM · SIGXCPU

반복 동작: 숫자를 먼저 예측한다 → 측정한다 → 예측과 다르면 왜 다른지 설명한다

실행 화면은 iTerm에서 Ubuntu 24.04(x86-64 · 커널 6.8) 실습 서버에 접속해 캡처한 실제 화면입니다.

관찰에서 측정으로

7주차 1차시: 관찰	7주차 2차시: 측정 · 실험
VmRSS가 VmSize보다 작다	RSS는 언제, 얼마씩 커지나? → pagetouch
STAT가 S 또는 R이다	CPU를 얼마나 쓰고, 얼마나 자주 잠드나? → pstat · cpu_wait
ps에 Z가 보인다	좀비는 언제 생기고 누가 치우나? 부모가 먼저 죽으면? → reap_show
NOFILE soft 한계는 1024다	한계를 넘으면 정확히 무엇이 일어나나? → limit_lab

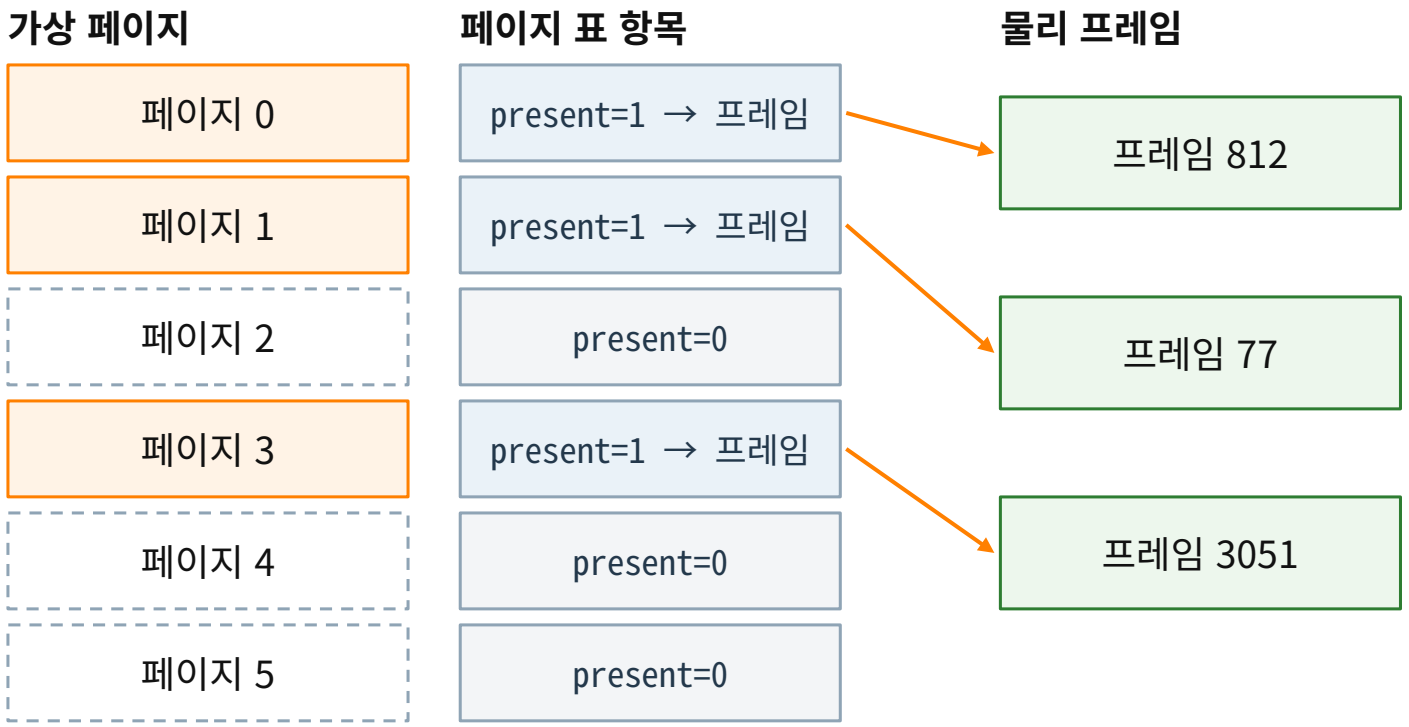
각 질문에 답하는 실습 프로그램이 하나씩 있습니다.

1부

메모리: 요구 페이징을 센다

페이지 폴트 수와 VmRSS로 메모리가 올라오는 순간을 측정합니다.

페이지와 페이지 표 항목



2 · 4 · 5번을 처음 건드리면 페이지 폴트 → 커널이 그때 프레임을 붙인다

페이지

4096바이트 단위로 주소를 나누고 이 단위로 연결한다.

present 비트

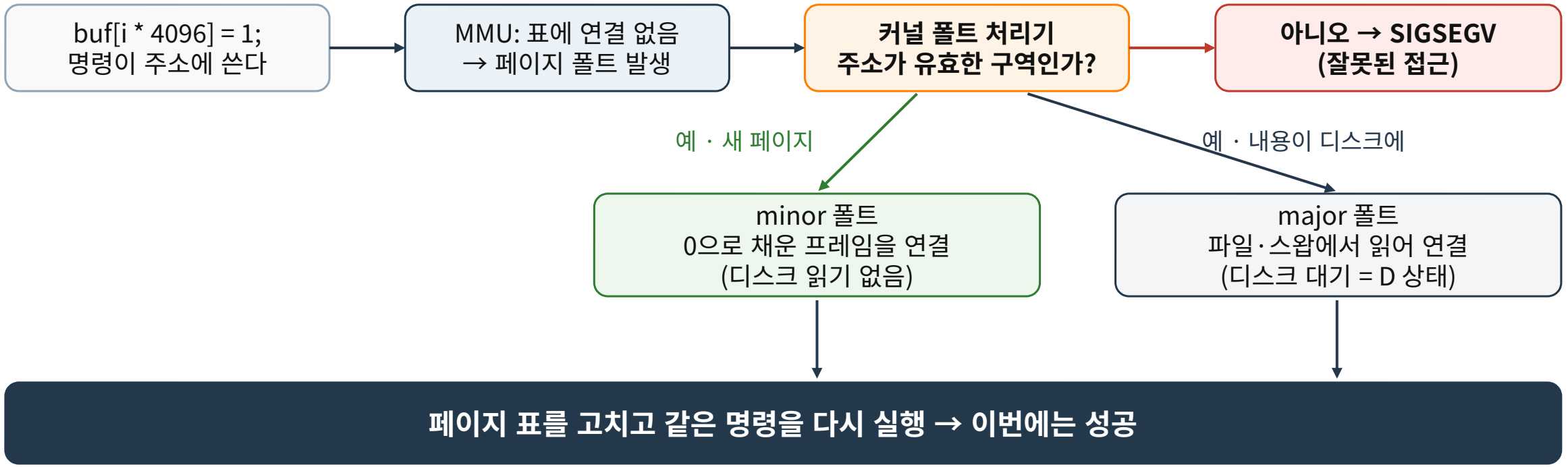
물리 프레임에 연결되었는지를 표시한다. 꺼져 있으면 폴트.

예약 ≠ 연결

malloc은 범위를 예약할 뿐, 연결은 처음 건드릴 때 한다.

실습 서버: getconf PAGESIZE → 4096. macOS(Apple 실리콘)는 16384입니다.

페이지 폴트가 처리되는 순서



폴트는 오류가 아니라 정상 동작입니다. 프로그램은 폴트가 일어난 줄도 모른 채 같은 명령을 다시 실행합니다.

세 가지 폴트

minor (ru_minflt)

디스크 없이 처리된다. 새 페이지,
이미 캐시에 있는 파일 페이지.

major (ru_majflt)

디스크를 읽어야 한다. 처음 실행한
큰 프로그램, 스왑에서 돌아온 페이지.

잘못된 접근

매핑 밖이거나 권한 위반이다.
NULL 역참조, 코드에 쓰기 → SIGSEGV.

폴트 수는 getrusage(표준 API)로 셀 수 있습니다.

pagetouch: 페이지마다 한 바이트

```
report("start");
volatile char *buf = malloc(size);      /* 예약만 */
report("after_malloc");
for (size_t i = 0; i < pages / 2; i++)
    buf[i * (size_t)page] = 1;          /* 절반을 처음 건드림 */
report("touch_half");
for (size_t i = pages / 2; i < pages; i++)
    buf[i * (size_t)page] = 1;          /* 나머지 절반 */
report("touch_all");
for (size_t i = 0; i < pages; i++)
    buf[i * (size_t)page] = 2;          /* 두 번째 접근 */
report("touch_again");
free((void *)buf);
report("after_free");
```

완성 예제: labs/class07-02/pagetouch.c 70–88행

한 페이지에 한 바이트

page 간격으로 쓰면 바이트 수가 아니라 페이지 수만큼 폴트가 난다.

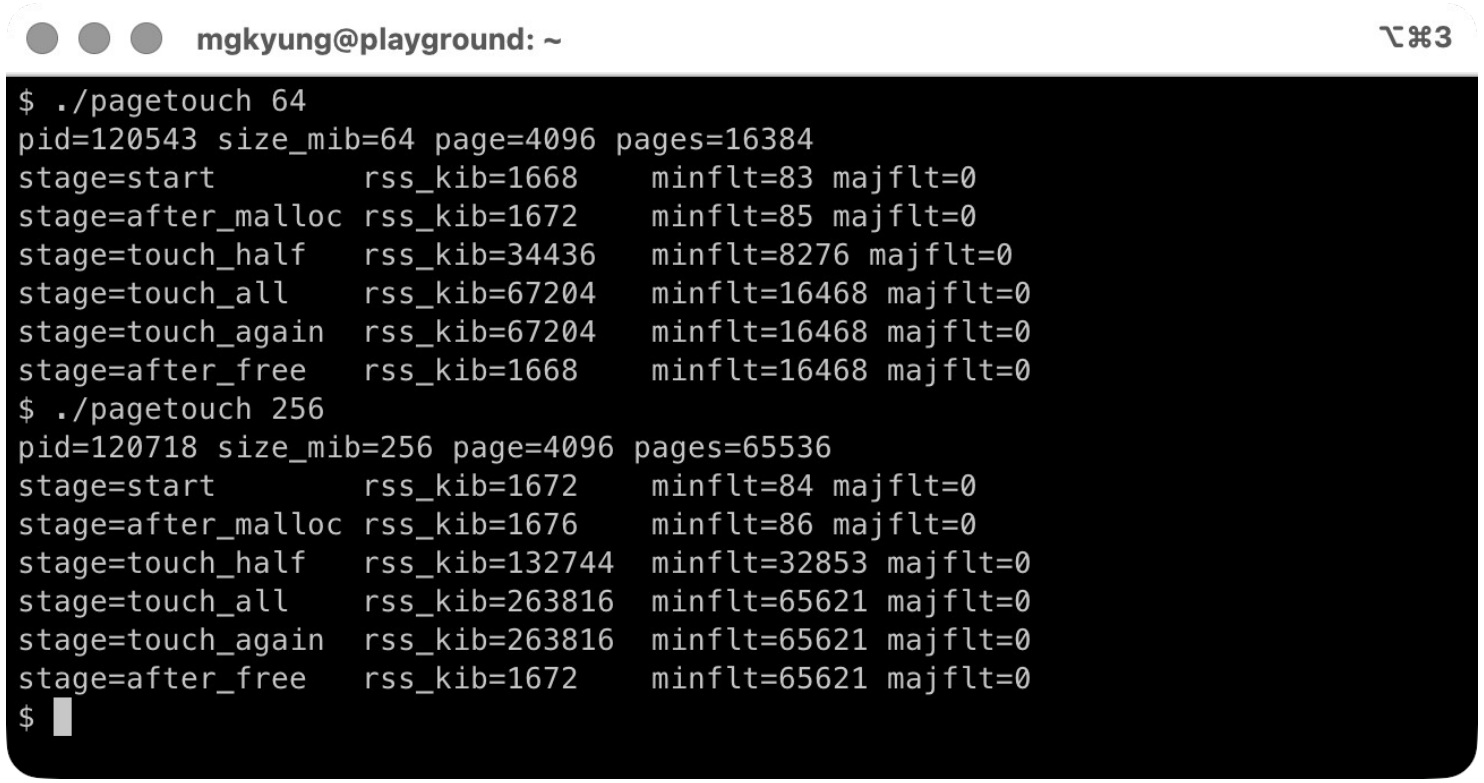
report

getrusage의 minflt·majflt와 /proc/self/status의 VmRSS를 찍는다.

volatile

free 직전의 쓰기를 컴파일러가 쓸모없다고 지우지 못하게 한다.

pagetouch 실행 화면



iTerm · Ubuntu 24.04 실습 서버 실제 화면

malloc 직후

64 MiB를 받았는데 RSS +4 KiB,
폴트 +2. 예약만 한 것이다.

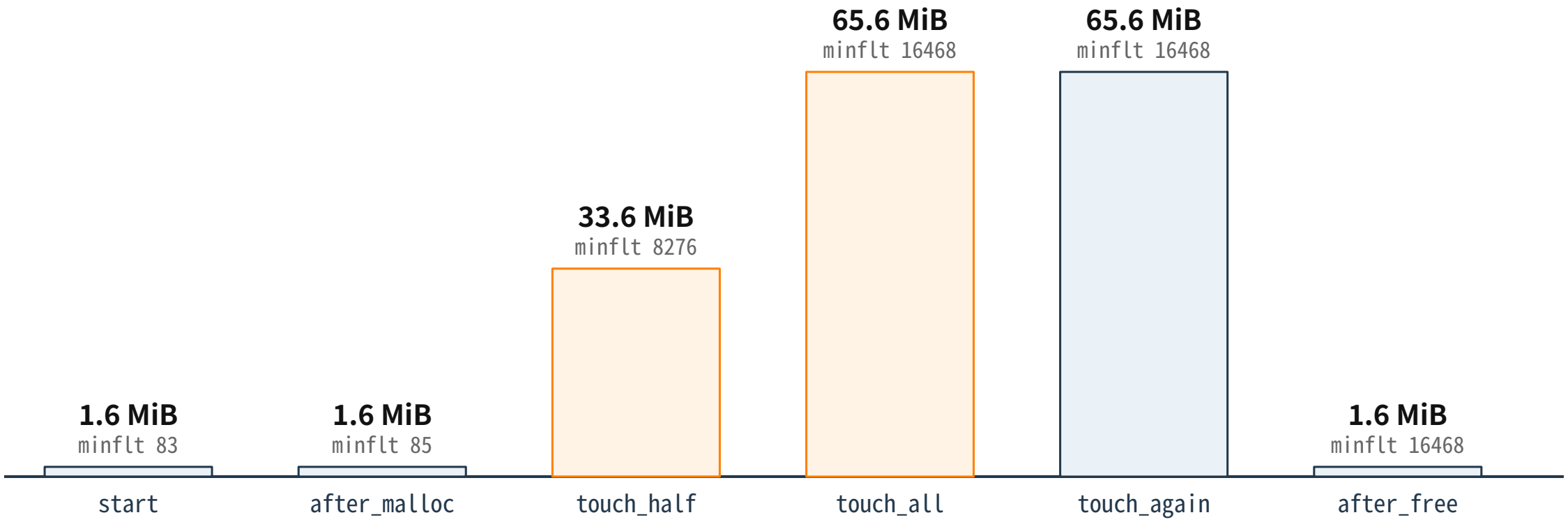
처음 건드릴 때

64 MiB: 폴트 합 16383 ≈ 16384쪽.
256 MiB: 65535 ≈ 65536쪽.

두 번째 접근

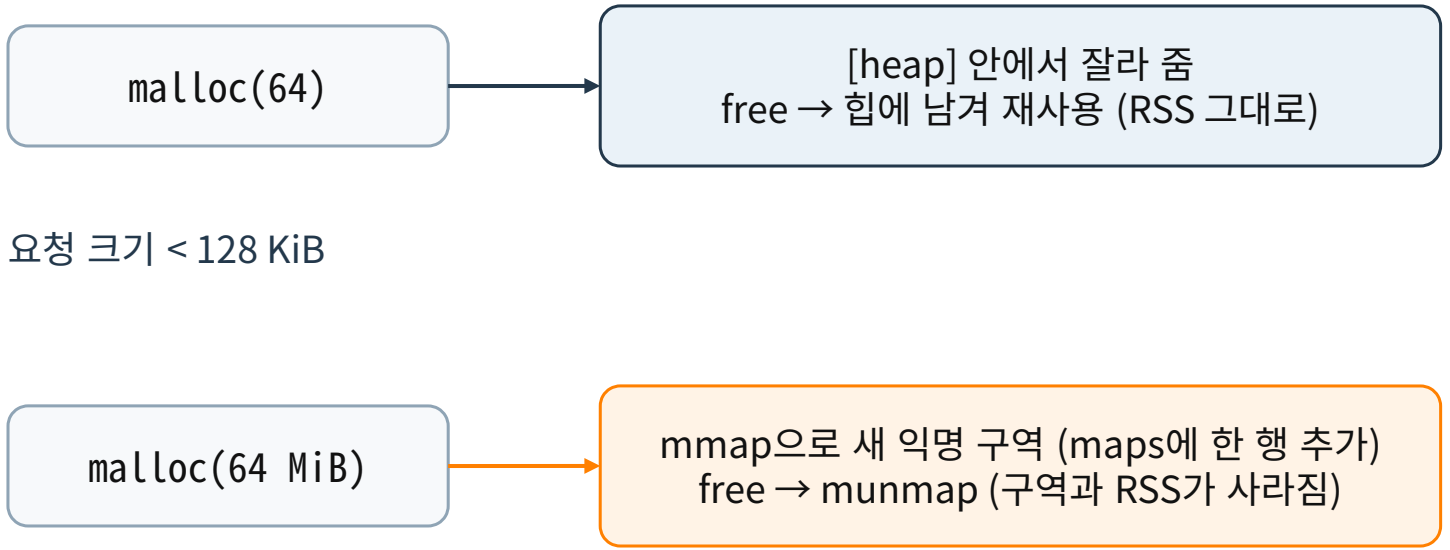
폴트 0, RSS 그대로. 이미
연결된 페이지는 폴트가 없다.

막대로 본 요구 페이징



VmRSS(KiB)를 MiB로 바꾸어 그렸습니다. 막대 위 숫자는 누적 minflt입니다.

free 뒤에 RSS가 줄어든 이유



큰 요청은 mmap

기본 128 KiB 이상은 별도
익명 구역으로 받는다.

free = munmap

구역째 커널에 돌려주므로
그 페이지가 RSS에서 빠진다.

작은 요청은 힙

free해도 힙에 남겨 재사용한다.
RSS가 바로 줄지 않는다.

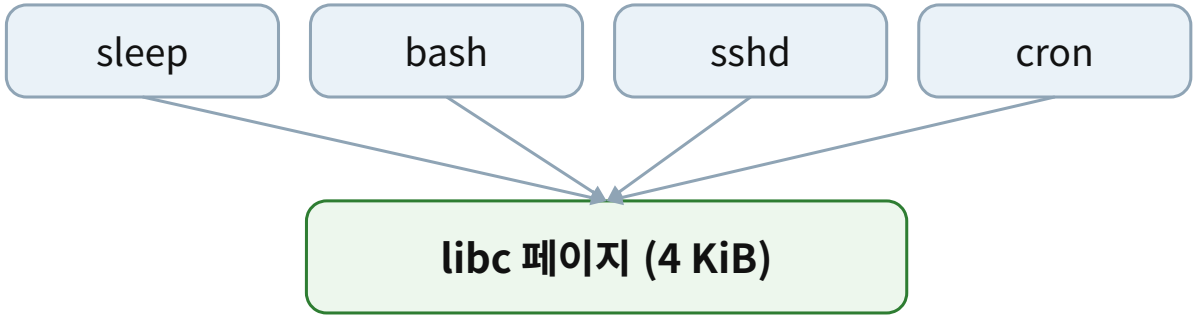
기준값은 mallopt(3)의 M_MMAP_THRESHOLD(기본 128 KiB, 64비트에서 동적 조정 상한 32 MiB)입니다. glibc 구현의 동작이며 C 표준의 계약은 아닙니다.

RSS와 PSS: 공유 페이지 나누기

mgkyung@playground: ~ ⌥⌘3

```
$ sleep 60 & sleep 0.2
[1] 120884
$ grep -E '^(Rss|Pss|Shared|Private)' /proc/$!/smaps_rollup
Rss:                2124 kB
Pss:                 204 kB
Pss_Dirty:           108 kB
Pss_Anon:            108 kB
Pss_File:             96 kB
Pss_Shmem:             0 kB
Shared_Clean:       2016 kB
Shared_Dirty:          0 kB
Private_Clean:         0 kB
Private_Dirty:       108 kB
Shared_Hugetlb:        0 kB
Private_Hugetlb:       0 kB
$ grep -E '^(VmRSS|RssAnon|RssFile)' /proc/$!/status
VmRSS:      2124 kB
RssAnon:             108 kB
RssFile:            2016 kB
$
```

libc 페이지 하나를 N개 프로세스가 공유하면



RSS: 프로세스마다 4 KiB
(모두 더하면 16 KiB)

PSS: 프로세스마다 4/N = 1 KiB
(모두 더하면 4 KiB)

sleep의 RSS 2124 kB 중 2016 kB는 다른 프로세스와
함께 쓰는 파일 페이지 · 이 프로세스 몫(PSS)은 204 kB

iTerm · Ubuntu 24.04 실습 서버 실제 화면 (sleep 60) · man 5 proc_pid_smaps

malloc 성공 ≠ 메모리 확보

과잉 약속

Linux 기본(overcommit_memory=0)은
실제 여유보다 큰 예약도 허락한다.

진짜 부족은 나중에

페이지를 건드릴 때 프레임이 없으면
그때 문제가 된다.

OOM killer

커널이 프로세스 하나를 골라
SIGKILL로 끝내 메모리를 되찾는다.

실습 서버: cat /proc/sys/vm/overcommit_memory → 0

직접 확인 1: 메모리

```
./pagetouch 64
./pagetouch 256                          # 폴트 수를 먼저 예측
getconf PAGESIZE

../class07-01/addr_show 30 &        # 7주차 1차시 프로그램
grep -E '^(Rss|Pss|Shared|Private)' /proc/$!/smaps_rollup
grep -E '^(VmRSS|RssAnon|RssFile)' /proc/$!/status
wait

cat /proc/sys/vm/overcommit_memory
```

기록은 labs/class07-02/REPORT.md의 실습 1 칸에 합니다.

예측 먼저

256 MiB면 touch_all까지 minflt가 몇 늘지 계산한다.

macOS 비교

macOS는 페이지가 16 KiB라 폴트 수가 1/4이 된다.

PSS 비교

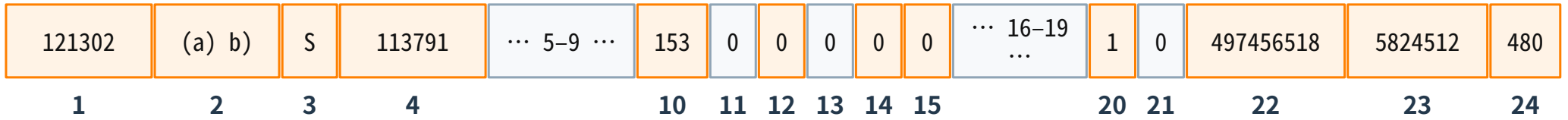
RSS · PSS · Private를 적고 차이가 어디서 오는지 설명한다.

2부

상태와 CPU: /proc/PID/stat 해석

한 줄짜리 stat를 파싱해 %CPU와 문맥 교환을 계산합니다.

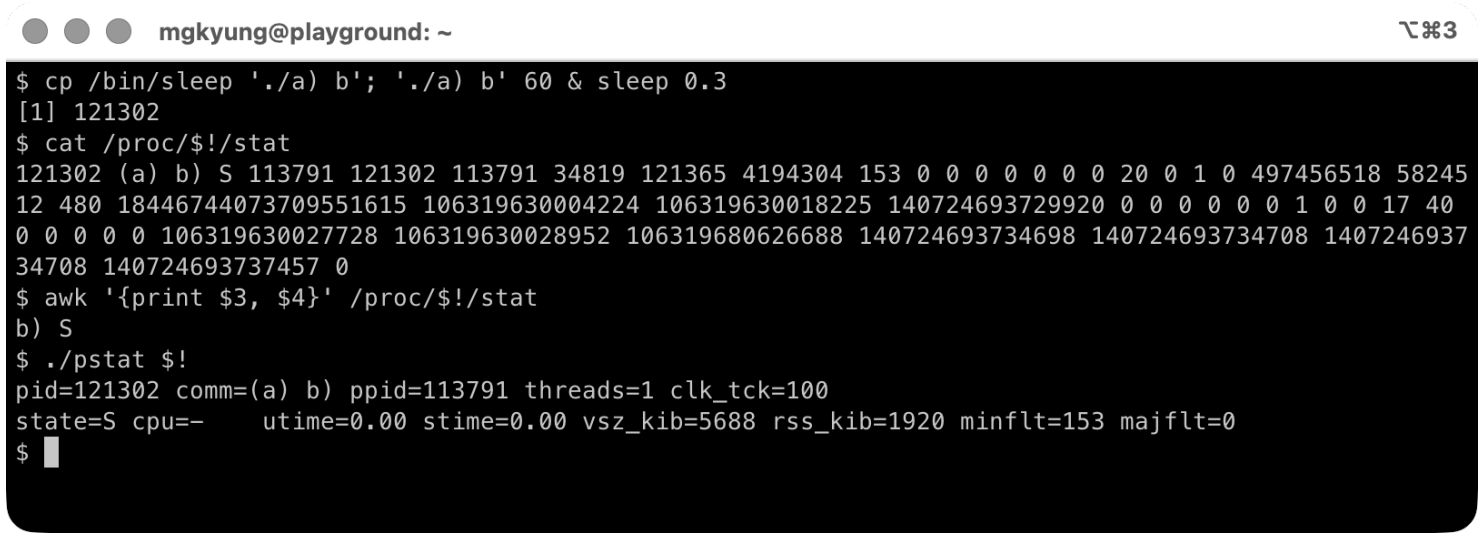
/proc/PID/stat 한 줄의 필드



3 state	상태 문자 (ps STAT의 첫 글자)
10 · 12 minflt · majflt	누적 폴트 수 (getrusage와 같은 카운터)
14 · 15 utime · stime	사용자·커널 모드 CPU 시간 · 단위: 클럭 틱
22 starttime	부팅 후 몇 틱에 시작했나
23 vsize · 24 rss	가상 크기(바이트) · 상주 페이지 수(쪽)

다음 장 화면의 실제 줄(실행 파일 이름이 "a) b"인 sleep) · 필드 번호는 man 5 proc_pid_stat 기준(1부터)

comm 함정: 괄호와 공백



comm = 첫 (와 마지막) 사이 · 나머지 필드 = 마지막) 뒤에서 공백으로 나눔

iTerm · Ubuntu 24.04 실습 서버 실제 화면

이름은 자유롭다

실행 파일 이름에 공백과
괄호가 들어갈 수 있다.

단순 분할은 틀린다

awk의 $3이 "b)"가 되었다.
실제 상태는 S다.

마지막)를 찾는다

pstat은 strrchr(line, ')')로
자르므로 state=S를 읽었다.

틱에서 초로, 초에서 %CPU로

```
state=R cpu=100% utime=2.19 stime=0.00 ...
state=R cpu=100% utime=3.19 stime=0.00 ...
state=R cpu=100% utime=4.19 stime=0.00 ...
```

① 틱 → 초	utime 필드 219 ÷ CLK_TCK 100 = 2.19초
② 1초 뒤 다시 읽음	utime 319 → 차이 Δ = 319 − 219 = 100틱
③ %CPU	100 × Δ ÷ (CLK_TCK × 간격 1초) = 100%

클럭 틱

CPU 시간의 단위다.
sysconf(_SC_CLK_TCK)로 얻는다.

차이로 계산

누적값 둘의 차이를 간격으로
나눠야 "지금"의 사용률이 된다.

100% 넘을 수 있다

멀티스레드는 코어 여러 개를
쓰므로 200%도 가능하다.

실습 서버: getconf CLK_TCK → 100 · 다음 장 pstat 화면의 값

pstat: stat 한 줄 파싱

```
char *open = strchr(line, '(');
char *close = strrchr(line, ')');       /* 마지막 ) */
if (open == NULL || close == NULL || close < open)
    return -1;
memcpy(ps->comm, open + 1, n);          /* 이름 복사 */
/* 필드 3(state)부터: %*는 읽고 버린다 */
int got = sscanf(close + 2,
    "%c %ld %*d %*d %*d %*d %*u %lu %*u %lu %*u %lu %lu "
    "%*d %*d %*d %*d %ld %*d %*u %lu %ld",
    &ps->state, &ps->ppid, &ps->minflt, &ps->majflt,
    &ps->utime, &ps->stime, &ps->threads, &ps->vsize,
    &ps->rss_pages);
return got == 9 ? 0 : -1;
```

완성 예제: labs/class07-02/pstat.c 33–49행

strrchr

이름 안에)가 있어도 마지막)는 필드 경계다.

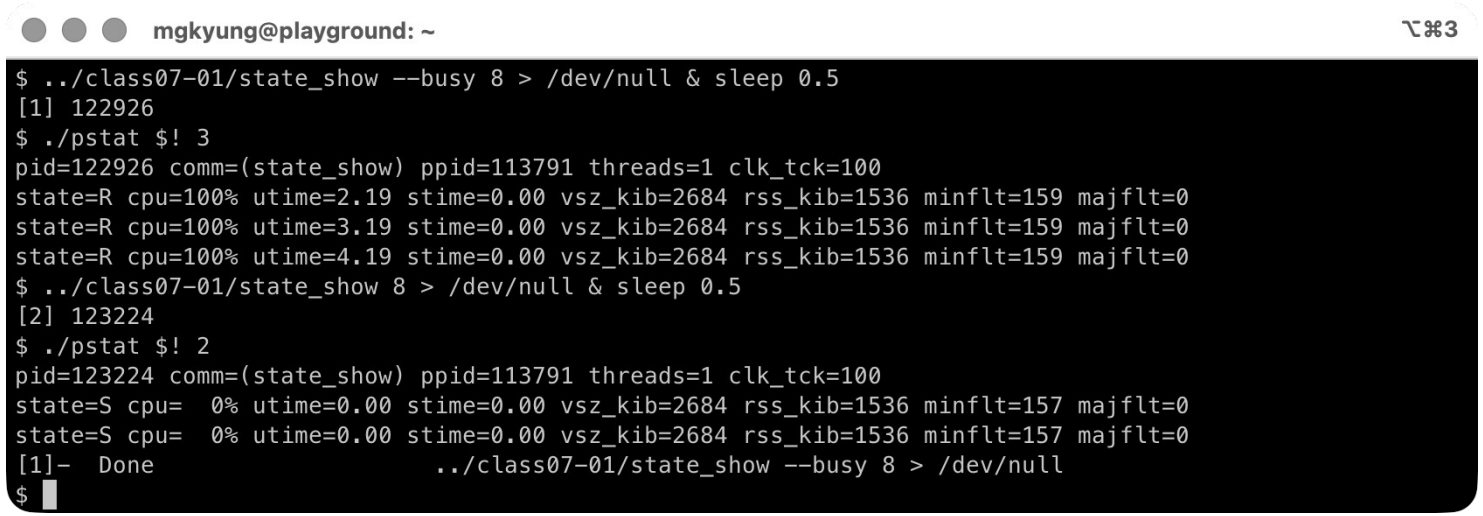
%*로 건너뛰기

필요 없는 필드는 읽고 버려 필드 번호를 맞춘다.

개수 검사

sscanf 반환값이 9가 아니면 형식이 예상과 다른 것이다.

pstat 실행 화면: R과 S



iTerm · Ubuntu 24.04 실습 서버 실제 화면

busy

R · 100% · utime만 1초에 1.00씩 는다. 커널 시간은 0.

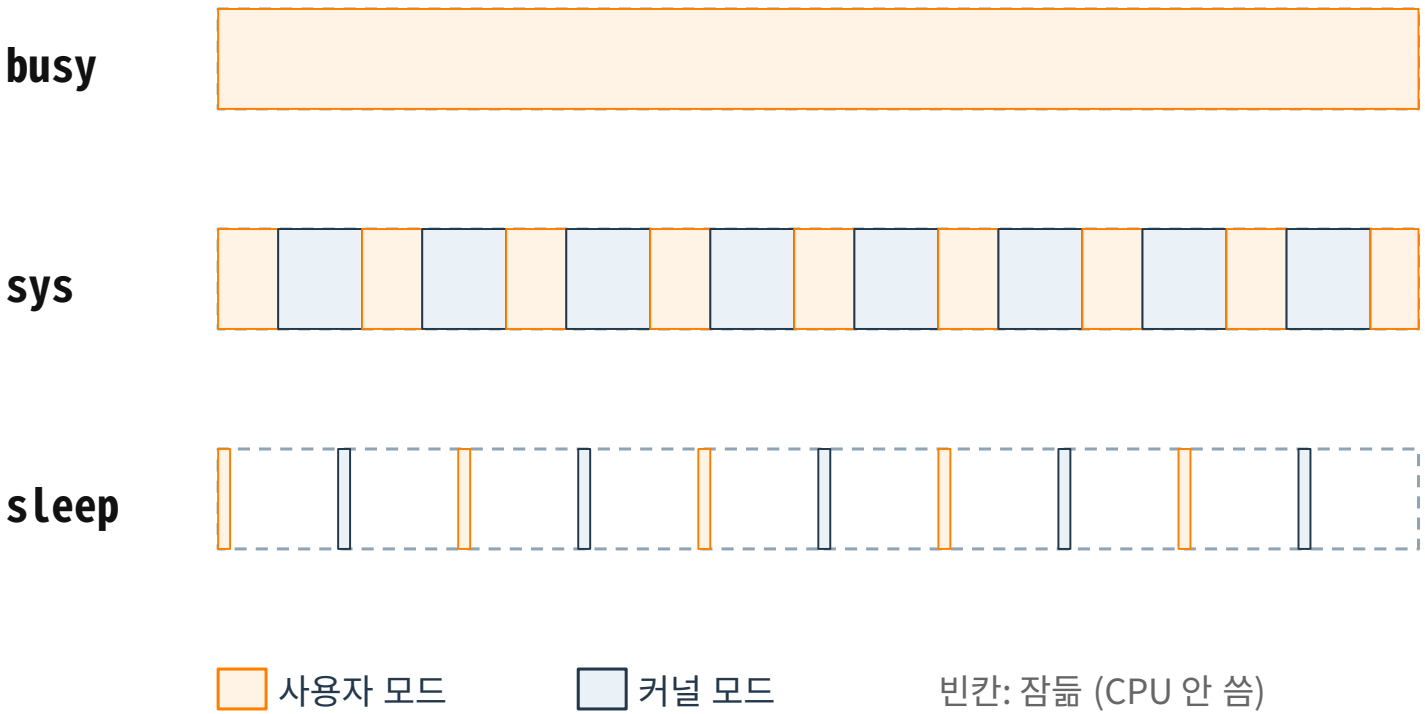
sleep

S · 0% · CPU 시간이 늘지 않는다. 잠든 동안 틱이 쌓이지 않는다.

변하지 않는 값

rss · minflt가 그대로다. 새 페이지를 건드리지 않았다.

user 시간과 system 시간



user (utime)

내 코드가 CPU에서 계산한 시간.
루프, 수식, 문자열 처리.

sys (stime)

내 요청으로 커널이 일한 시간.
write · read · open 같은 호출.

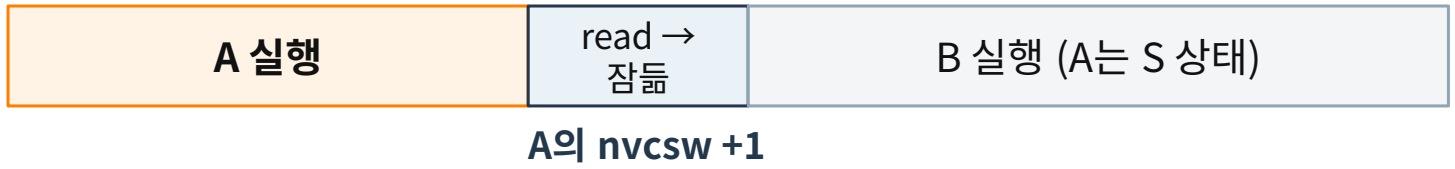
둘 다 아님

잠든 시간은 CPU 시간이 아니다.
벽시계 시간만 흐른다.

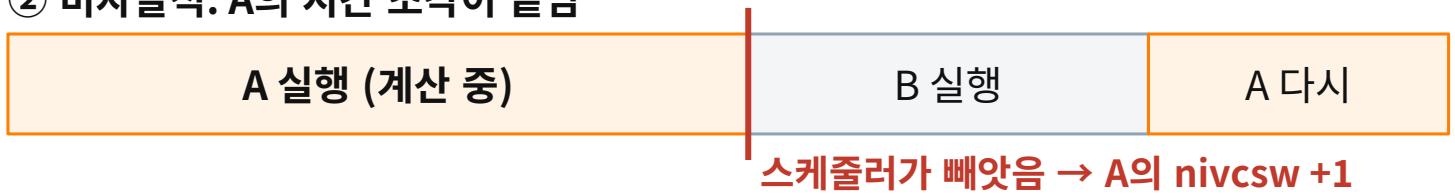
time 명령의 user · sys · real이 같은 구분입니다. real(벽시계) ≥ user + sys 이면 기다린 시간이 있었다는 뜻입니다(단일 스레드).

문맥 교환: 자발적과 비자발적

① 자발적: A가 read에서 잠듦



② 비자발적: A의 시간 조각이 끝남



A는 ①에서 S로 기다리고, ②에서는 R인 채로 줄을 선다

자발적 (nvcsw)

기다릴 일이 생겨 스스로 CPU를 내놓았다. 잠·I/O 대기.

비자발적 (nivcsw)

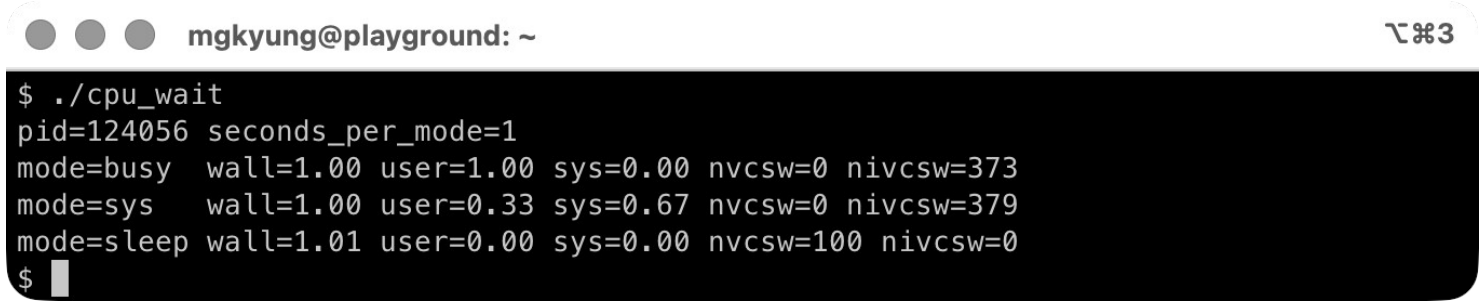
더 하고 싶었지만 빼앗겼다. 시간 조각 끝 · 우선순위.

읽는 법

nvcsw가 크면 기다리는 프로그램, nivcsw가 크면 CPU 경쟁 중.

getrusage의 ru_nvcsw · ru_nivcsw, /proc/PID/status의 voluntary · nonvoluntary_ctxt_switches가 같은 카운터입니다.

cpu_wait: 같은 1초, 다른 숫자




```
/* sys 모드의 반복 본문 */
if (write(devnull, &byte, 1) == -1) { ... }
/* sleep 모드의 반복 본문 */
usleep(10000);                          /* 10ms */
```

iTerm · Ubuntu 24.04 실습 서버 실제 화면

busy

user 1.00, nvcsw 0.
한 번도 스스로 양보하지 않았다.

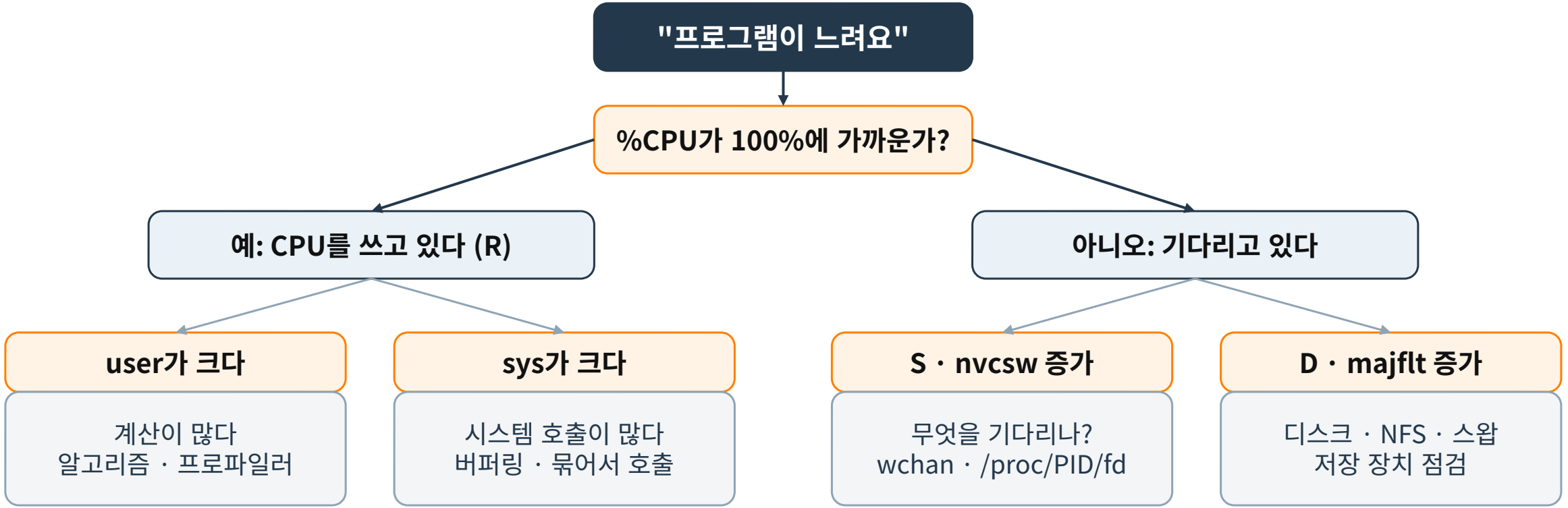
sys

sys 0.67. 1바이트 write를
반복하면 시간 대부분이 커널이다.

sleep

CPU 0초, nvcsw 100.
10ms 잠을 100번 = 자발적 100번.

"느리다"를 숫자로 진단하기



7주차 1차시의 상태 문자와 오늘의 user · sys · 교환 횟수 · 폴트 수가 함께 진단 도구가 됩니다.

wchan과 load average



iTerm · Ubuntu 24.04 실습 서버 실제 화면

wchan: 잠든 자리

sleep 30은 hrtimer_nanosleep 에서 잔다. 무엇을 기다리나의 증거.

load average

R과 D인 작업 수의 1·5·15분 평균이다. Linux는 D도 센다.

코어 수와 비교

48코어에서 6.28은 한가하다. 넷째 칸 29/4234는 지금/전체.

직접 확인 2: 상태와 CPU

```
./pstat self
../class07-01/state_show --busy 10 > /dev/null &
./pstat $! 5                          # %CPU · utime 변화
wait
../class07-01/state_show 10 > /dev/null &
./pstat $! 3 ; wait

cp /bin/sleep "./a) b" ; "./a) b" 5 &
awk '{print $3, $4}' /proc/$!/stat  # 함정
./pstat $!                            # 바른 파싱
wait ; rm "./a) b"

./cpu_wait ; cat /proc/loadavg
```

기록은 REPORT.md의 실습 2 칸에 합니다.

pstat 두 모드

R · S에서 utime · stime · %CPU가 어떻게 다른지 적는다.

comm 함정

awk 결과와 pstat 결과를 나란히 적고 원인을 설명한다.

cpu_wait

세 모드의 user · sys · nvcsw를 예측한 뒤 실행해 비교한다.

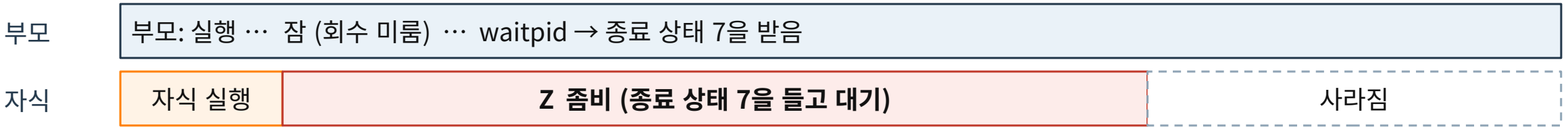
3부

수명: 좀비와 고아

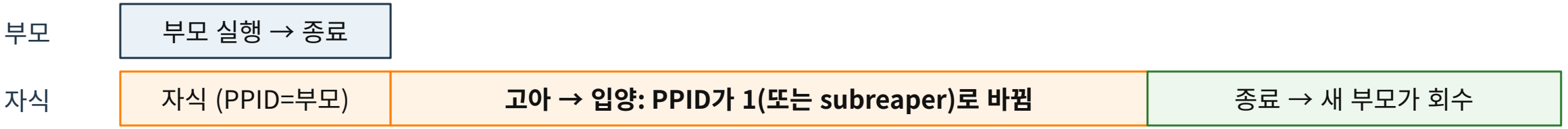
종료와 회수 사이, 부모가 먼저 끝난 경우를 직접 만듭니다.

부모와 자식, 누가 먼저 끝나나

경우 1: 자식이 먼저 끝난다 (reap_show zombie)



경우 2: 부모가 먼저 끝난다 (reap_show orphan)



어느 경우든 종료 상태는 누군가 회수한다. 문제는 살아 있는 부모가 회수하지 않을 때뿐이다.

fork와 waitpid의 정식 계약은 10주차에서 다룹니다. 오늘은 reap_show가 두 경우를 만들어 주는 관찰 도구입니다.

reap_show zombie: 회수를 미루기

```
pid_t child = fork();
if (child == -1) {
    perror("fork");
    return 1;
}
if (child == 0)
    _exit(7);                /* 자식: 즉시 종료 상태 7 */
printf("parent=%ld child=%ld (...)\n",
       (long)getpid(), (long)child);
unsigned int left = (unsigned int)seconds;
while (left != 0)
    left = sleep(left);      /* 부모: 회수를 미룬다 */
int status;
if (waitpid(child, &status, 0) == -1) { ... }
if (WIFEXITED(status))
    printf("reaped child=%ld exit_status=%d\n",
           (long)child, WEXITSTATUS(status));
```

완성 예제: labs/class07-02/reap_show.c 26–48행

fork의 두 반환

자식에게는 0, 부모에게는
자식 PID를 돌려준다.

_exit(7)

자식은 stdio 정리 없이
종료 상태 7로 즉시 끝난다.

waitpid

종료 상태를 읽어 가면
좀비 항목이 지워진다.

좀비 실제 화면



iTerm · Ubuntu 24.04 실습 서버 실제 화면

Z와 <defunct>

명령 줄 대신 [이름]과 <defunct>가 표시된다.

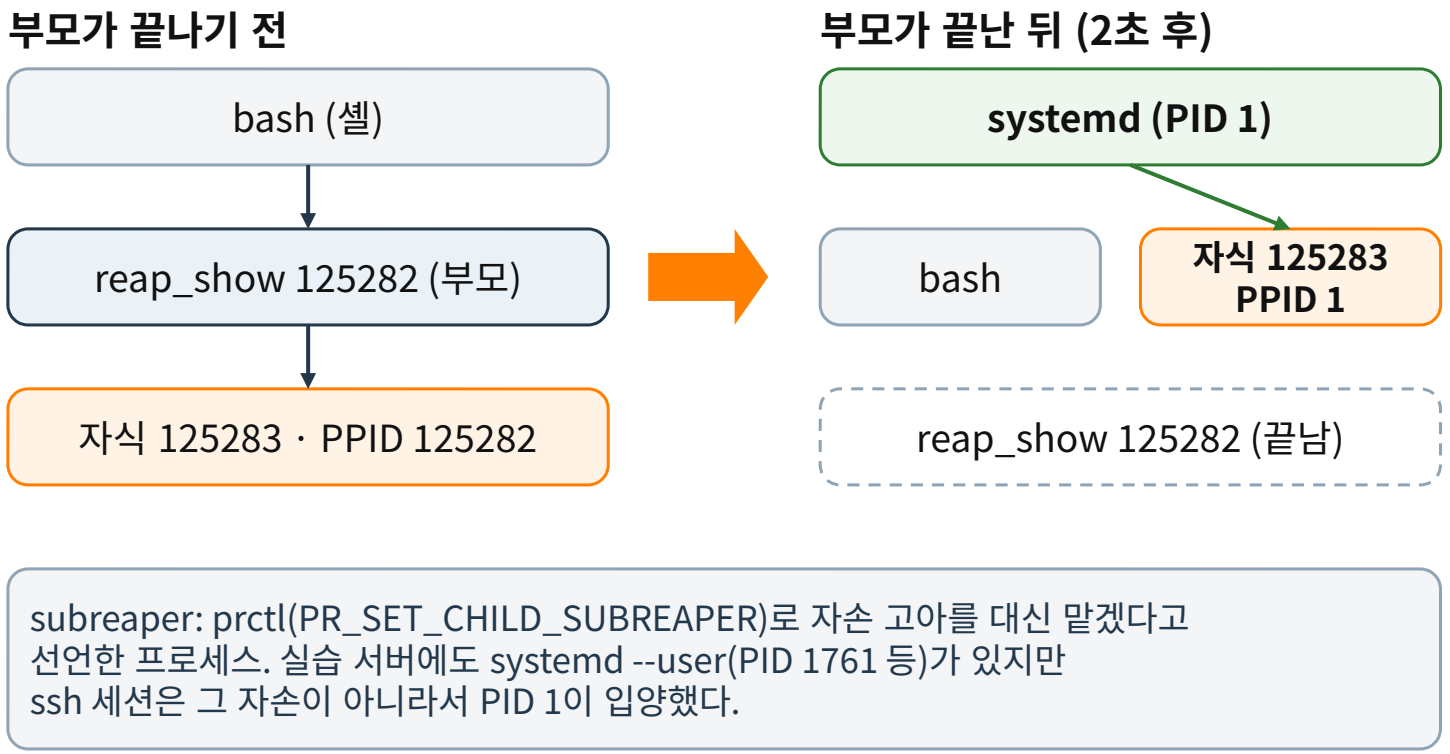
PPID

좀비의 PPID가 부모 reap_show를 가리킨다.

회수 후

6초 뒤 waitpid가 7을 받고 좀비가 사라졌다.

고아와 입양



subreaper: prctl(PR_SET_CHILD_SUBREAPER)로 자손 고아를 대신 맡겠다고 선언한 프로세스. 실습 서버에도 systemd --user(PID 1761 등)가 있지만 ssh 세션은 그 자손이 아니라서 PID 1이 입양했다.

입양

부모가 끝나면 커널이 자식의 PPID를 바꿔 준다.

누가 입양하나

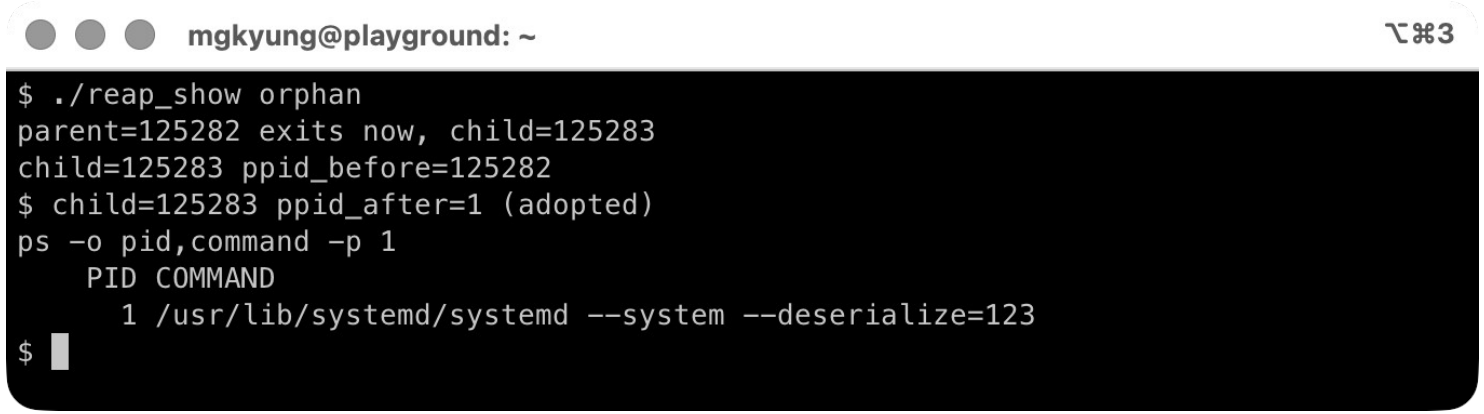
가장 가까운 subreaper 조상, 없으면 PID 1이다.

회수 보장

입양한 쪽이 wait하므로 고아는 좀비로 쌓이지 않는다.

실습 서버(ssh 접속)에서는 PPID가 1이 되었습니다. 데스크톱 세션에서는 systemd --user가 입양할 수 있습니다.

고아 실제 화면



iTerm · Ubuntu 24.04 실습 서버 실제 화면

프롬프트가 먼저

셸은 부모만 기다린다.
자식 출력이 프롬프트 뒤에 붙었다.

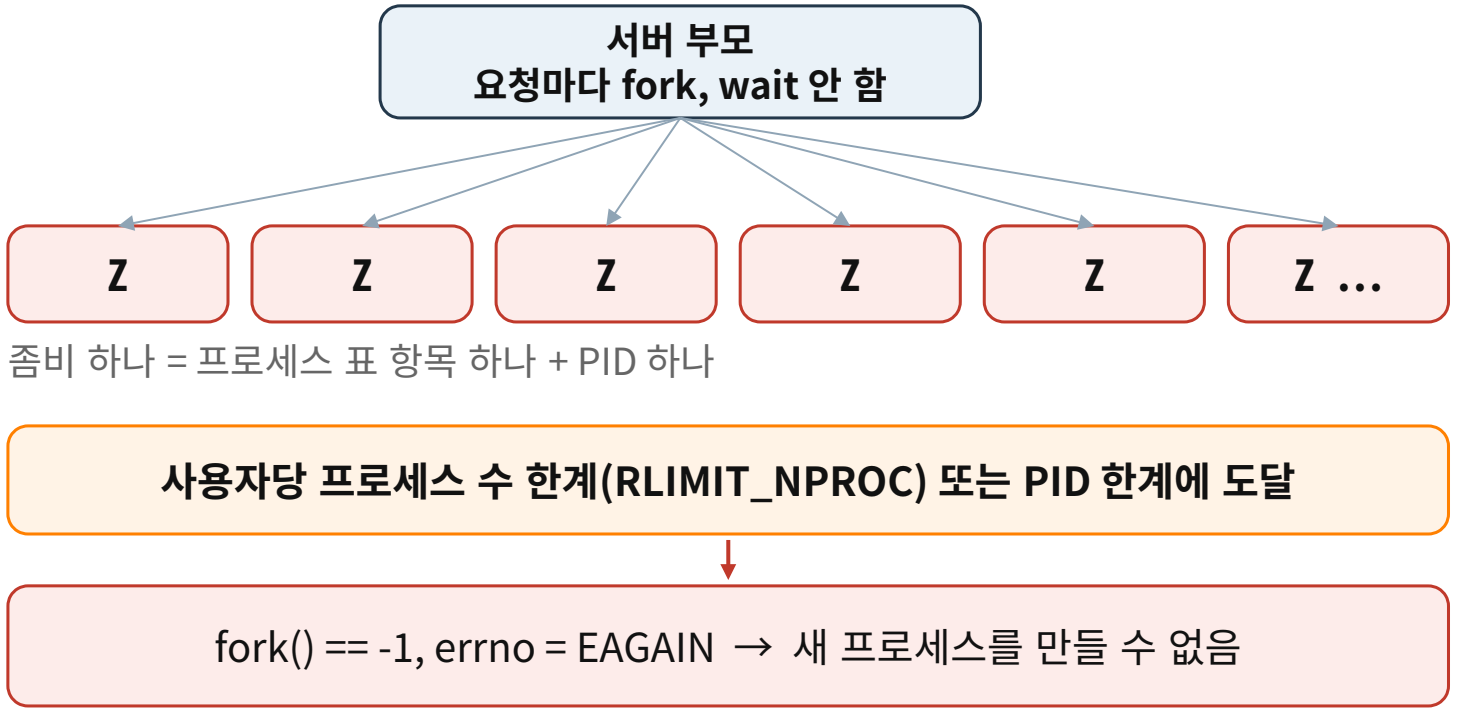
PPID 1

2초 뒤 자식의 부모가
systemd(PID 1)로 바뀌었다.

getppid

같은 호출이 시간에 따라
다른 값을 돌려줄 수 있다.

좀비가 쌓이는 서버



한 항목은 작지만

좀비는 메모리를 거의 안 쓰지만
PID 하나를 계속 차지한다.

한계에 닿으면

fork가 EAGAIN으로 실패한다.
새 요청을 처리할 수 없다.

고칠 곳은 부모

waitpid로 회수한다(10주차).
SIGCHLD로 알림 받기(9주차).

실습 서버 /proc/PID/limits: Max processes 1029894 (soft = hard)

직접 확인 3: 수명

```
./reap_show zombie 20 &
ps -o pid,ppid,stat,command --ppid $!     # Linux
ps -o pid,ppid,stat,command | grep defunct # macOS
head -3 /proc/<자식PID>/status
kill -9 <자식PID>          # 좀비는 사라지는가?
wait

./reap_show orphan ; sleep 3
setsid ./reap_show orphan ; sleep 3

ps -eo pid,ppid,stat,command | awk '$3 ~ /^Z/'
```

기록은 REPORT.md의 실습 3 칸에 합니다.

좀비에 kill

SIGKILL을 보내도 좀비가
남는 것을 확인하고 이유를 쓴다.

입양한 쪽

ppid_after 값과 그 PID의
명령을 ps로 확인한다.

시스템의 좀비

서버의 Z를 찾아 PPID로
부모를 확인한다.

4부

한계: 넘으면 무슨 일이

soft 한계를 낮추고 일부러 넘겨 결과를 확인합니다.

한계별로 넘으면 일어나는 일

한계 (RLIMIT_)	무엇을 세나	넘으면
NOFILE	열 수 있는 FD 번호의 상한	**open · socket 등이 -1, errno = EMFILE**
AS	가상 주소 공간 크기	**malloc · mmap이 실패, errno = ENOMEM**
CPU	누적 CPU 시간(초)	**soft: SIGXCPU (기본 동작: 종료) · hard: SIGKILL**
STACK	스택 크기	스택을 더 못 늘림 → SIGSEGV
NPROC	사용자의 프로세스 수	fork가 -1, errno = EAGAIN
FSIZE	만들 수 있는 파일 크기	write가 넘는 순간 SIGXFSZ
CORE	코어 덤프 파일 크기	0이면 코어 파일을 만들지 않음

man 2 getrlimit · 굵은 행 세 개를 limit_lab으로 실험합니다.

limit_lab: soft를 낮추고 넘기기

```
static long long lower_soft(int resource, rlim_t value)
{
    struct rlimit lim;
    if (getrlimit(resource, &lim) == -1) { ... }
    if (lim.rlim_max != RLIM_INFINITY && value > lim.rlim_max)
        value = lim.rlim_max;   /* soft는 hard를 넘을 수 없다 */
    lim.rlim_cur = value;
    if (setrlimit(resource, &lim) == -1) { ... }
    return (long long)value;    /* 실제로 적용한 값 */
}

/* nofile 모드: 실패할 때까지 연다 */
while (count < 1024) {
    int fd = open("/dev/null", O_RDONLY);
    if (fd == -1) { printf(... errno ...); break; }
    fds[count++] = fd;
}
```

완성 예제: labs/class07-02/limit_lab.c 19–51행

읽고 고치고 쓰기

getrlimit로 현재 쌍을 읽고 soft만 바꿔 setrlimit한다.

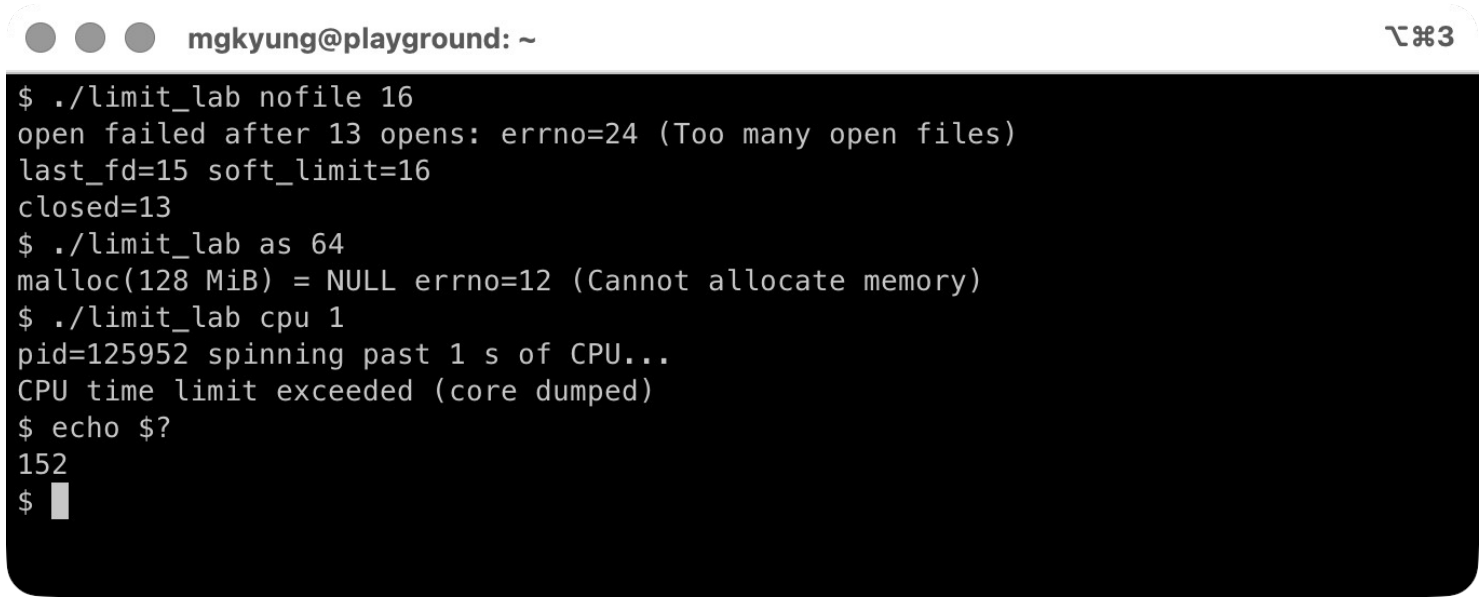
hard로 자르기

요청이 hard보다 크면 hard로 낮추고 실제 값을 돌려준다.

정리

연 FD를 배열에 모아 두었다가 실험 뒤 모두 닫는다.

세 한계를 넘긴 실제 화면



iTerm · Ubuntu 24.04 실습 서버 실제 화면

NOFILE 16

13번 열고 EMFILE(24).
0 · 1 · 2 + 3~15 = 16개.

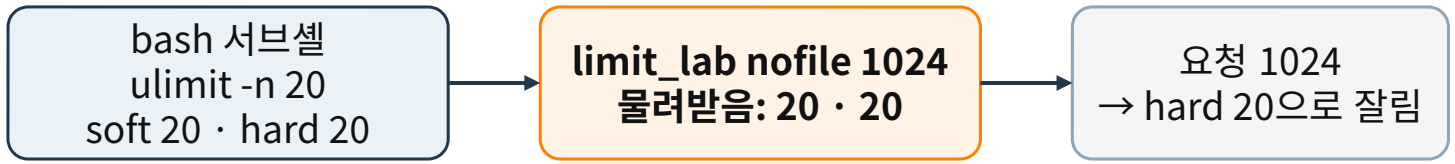
AS 64 MiB

128 MiB malloc이 NULL,
ENOMEM(12). 처리할 수 있다.

CPU 1초

SIGXCPU(24)로 끝났다.
종료 상태 152 = 128 + 24.

한계는 상속된다



mgkyung@playground: ~ ⌥⌘3

```
$ ( ulimit -n 20; ./limit_lab nofile 1024; ../class07-01/procdump | grep nofile )
open failed after 17 opens: errno=24 (Too many open files)
last_fd=19 soft_limit=20
closed=17
rlimit_nofile_soft=20
rlimit_nofile_hard=20
$ ulimit -n
1024
$
```

ulimit은 셸 내장

셸 자신의 한계를 바꾼다.
그 뒤의 자식이 물려받는다.

bash의 ulimit -n

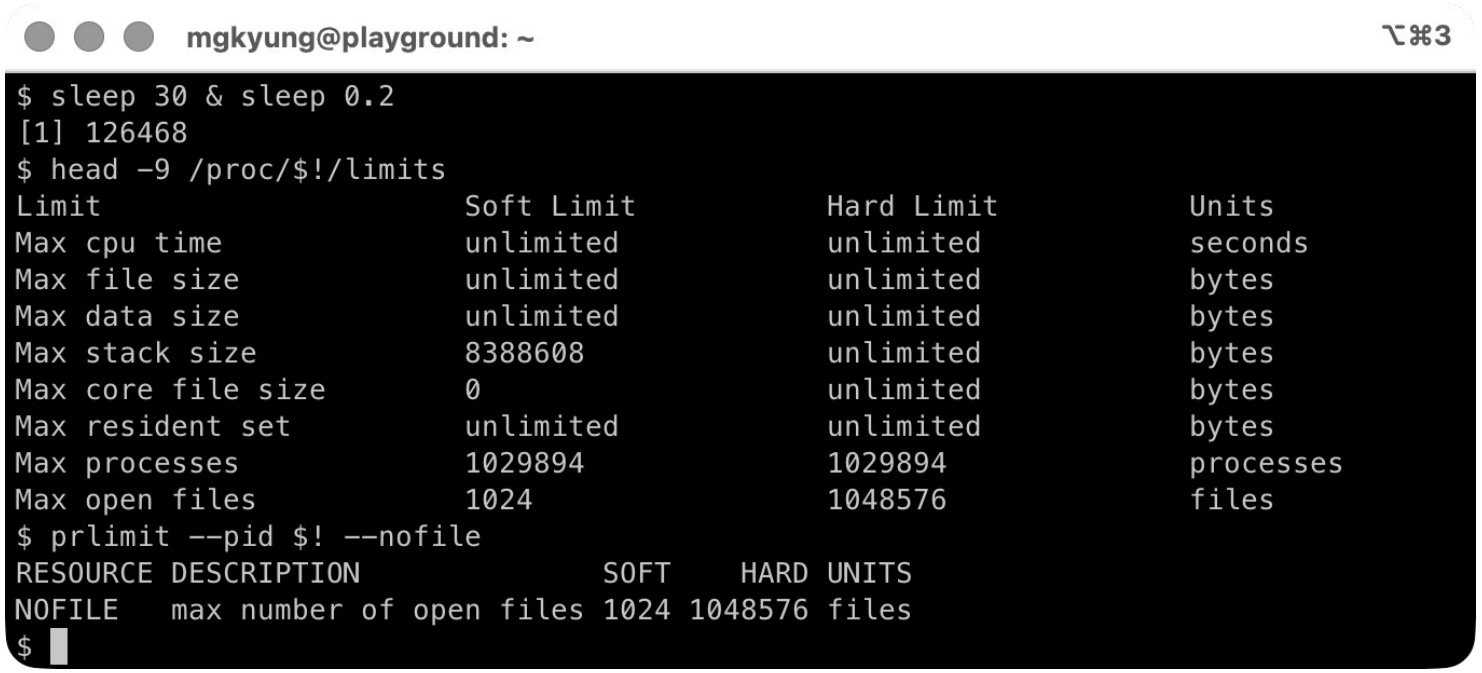
-S·-H 없이 쓰면 soft와 hard를 함께 바꾼다. 되돌릴 수 없다.

괄호 서브셸

(ulimit -n 20; ...) 처럼 쓰면 원래 셸은 그대로 남는다.

iTerm · Ubuntu 24.04 실습 서버 실제 화면 · 같은 명령이 macOS zsh에서는 soft만 바꿔 1024가 적용되었습니다.

/proc/PID/limits와 prlimit



iTerm · Ubuntu 24.04 실습 서버 실제 화면

다른 프로세스의 한계

/proc/PID/limits로 실행 중인
어떤 프로세스든 읽을 수 있다.

prlimit

util-linux 도구. 실행 중인
프로세스의 한계를 바꿀 수도 있다.

core 0

코어 파일 크기 soft가 0이라
기본으로는 코어 파일을 안 만든다.

직접 확인 4: 한계

```
./limit_lab nofile 16       # 몇 번 열까? 마지막 FD는?
./limit_lab nofile 4
./limit_lab as 64            # Linux
./limit_lab cpu 1 ; echo $?

( ulimit -n 20; ./limit_lab nofile 1024 )
ulimit -n                    # 원래 셸은 그대로인가?

head -9 /proc/$$/limits
prlimit --pid $$
```

기록은 REPORT.md의 실습 4 칸에 합니다.

예측

nofile N에서 열리는 개수와
마지막 FD 번호를 먼저 적는다.

종료 상태

cpu 모드의 $?를 시그널
번호로 해석한다(kill -l 152).

상속

서브셸 안팎의 ulimit -n을
비교해 적는다.

질문별 관찰 창과 측정값

질문	측정값 (표준 API)	측정값 (Linux /proc · 셸)
메모리는 언제 올라오나	getrusage: ru_minflt · ru_majflt	status VmRSS · smaps_rollup Pss
CPU를 얼마나 쓰나	getrusage: ru_utime · ru_stime	stat 14·15번 틱 차이 → %CPU
무엇을 기다리나	getrusage: ru_nvcsw · ru_nivcsw	ps wchan · /proc/loadavg
누가 회수하나	waitpid의 종료 상태	ps STAT Z · <defunct> · PPID
한계를 넘으면	errno (EMFILE · ENOMEM)	$? = 128 + 시그널 · /proc/PID/limits

숫자가 예측과 다르면 그 차이가 가장 많은 것을 가르쳐 줍니다.

정리

메모리: malloc은 예약이다. 처음 건드린 페이지만큼 폴트와 RSS가 는다.

공유: RSS는 공유 페이지를 중복해 세고, PSS는 나눠 센다.

CPU: stat의 utime·stime은 틱이고, 두 표본의 차이가 지금의 %CPU다.

수명: 자식이 먼저 끝나면 좀비, 부모가 먼저 끝나면 고아가 되어 입양된다.

한계: 넘으면 errno로 실패하거나 시그널로 끝난다. 한계는 상속된다.

읽기: man 5 proc_pid_stat · man 2 getrusage · man 2 getrlimit · 이어서: 9주차 시그널 → 10주차 fork·exec → 11주차 mmap

이것으로 할 수 있는 일

미니 top 만들기

/proc/[0-9]*/stat를 모두 읽어
%CPU · RSS 순으로 정렬하면
나만의 top이 된다 (pstat 확장).

메모리 예산 세우기

RSS 합 대신 PSS 합으로
서버 한 대에 프로세스를
몇 개 띄울 수 있는지 계산한다.

성능 문제 1차 진단

user · sys · nvcsw · majflt로
계산 · 호출 과다 · 대기 · 디스크
중 어디가 문제인지 가른다.

좀비 감시와 경보

주기적으로 Z 수를 세어
늘어나면 PPID로 부모를
찾아 알린다.

서비스 한계 설계

동시 연결 수에 맞춰 NOFILE을,
폭주 대비로 CPU · AS를
시작 스크립트에서 정한다.

실험 안전 장치

채점기 · 샌드박스에서 학생
코드에 CPU · AS · NPROC 한계를
걸어 무한 루프와 fork 폭탄을 막는다.

측정은 의견을 숫자로 바꾸는 일입니다.

pagetouch.c · 1–12행 · 헤더와 파일 주석

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  #include <string.h>
04  #include <sys/resource.h>
05  #include <unistd.h>
06
07  /* 요구 페이징(demand paging)을 숫자로 확인한다. 큰 블록을 malloc한
08     직후에는 VmRSS가 거의 늘지 않고, 페이지마다 한 바이트씩 써서
09     처음 건드릴 때마다 minor page fault가 일어나며 RSS가 늘어난다.
10     minflt는 getrusage(표준 API), VmRSS는 /proc/self/status(Linux)
11     에서 읽는다. /proc가 없으면 rss_kib=unavailable로 표시한다. */
12
```

labs/class07-02/pagetouch.c · 총 94행

1–5행 · 헤더

sys/resource.h는 getrusage,
unistd.h는 sysconf.

7–9행 · 가설

malloc 직후 RSS는 그대로,
처음 쓸 때 폴트와 함께 는다.

10–11행 · 두 출처

폴트 수는 표준 API, RSS는
/proc(Linux)에서 읽는다.

pagetouch.c · 13–28행 · rss_kib: VmRSS 한 줄 읽기

```
13  static long rss_kib(void)
14  {
15      FILE *fp = fopen("/proc/self/status", "r");
16      if (fp == NULL)
17          return -1;
18      char line[256];
19      long kib = -1;
20      while (fgets(line, sizeof line, fp) != NULL) {
21          if (strncmp(line, "VmRSS:", 6) == 0) {
22              kib = strtol(line + 6, NULL, 10);
23              break;
24          }
25      }
26      fclose(fp);
27      return kib;
28  }
```

labs/class07-02/pagetouch.c · 총 94행

15–17행 · 열기

/proc가 없으면 -1을 돌려 "알 수 없음"을 알린다.

20–25행 · 찾기

VmRSS:로 시작하는 행에서 숫자를 strtol로 읽는다.

26–27행 · 정리

fclose 후 KiB 값을 돌려준다.

pagetouch.c · 29–45행 · report: 한 단계 기록

```
29
30  static void report(const char *stage)
31  {
32      struct rusage ru;
33      if (getrusage(RUSAGE_SELF, &ru) == -1) {
34          perror("getrusage");
35          exit(1);
36      }
37      long kib = rss_kib();
38      if (kib < 0)
39          printf("stage=%-12s rss_kib=unavailable minflt=%ld majflt=%ld\n",
40                 stage, ru.ru_minflt, ru.ru_majflt);
41      else
42          printf("stage=%-12s rss_kib=%-7ld minflt=%ld majflt=%ld\n",
43                 stage, kib, ru.ru_minflt, ru.ru_majflt);
44  }
45
```

labs/class07-02/pagetouch.c · 총 94행

32–36행 · getrusage

minflt · majflt를 표준 API로 얻는다.

37행 · RSS

rss_kib로 지금 상주 크기를 얻는다.

38–43행 · 출력

RSS가 없으면 unavailable, 있으면 열을 맞춰 출력한다.

pagetouch.c · 46–65행 · 인자와 페이지 크기

```
46  int main(int argc, char **argv)
47  {
48      long mib = 64;
49      if (argc > 2) {
50          fprintf(stderr, "usage: %s [MiB 1-1024]\n", argv[0]);
51          return 2;
52      }
53      if (argc == 2) {
54          char *end = NULL;
55          mib = strtol(argv[1], &end, 10);
56          if (end == argv[1] || *end != '\0' || mib < 1 || mib > 1024) {
57              fprintf(stderr, "usage: %s [MiB 1-1024]\n", argv[0]);
58              return 2;
59          }
60      }
61      long page = sysconf(_SC_PAGESIZE);
62      if (page <= 0) {
63          perror("sysconf");
64          return 1;
65      }
```

labs/class07-02/pagetouch.c · 총 94행

48–60행 · MiB 인자

기본 64, 1–1024 범위의 정수만 받는다.

61–65행 · 페이지 크기

sysconf(_SC_PAGESIZE)로 실행 중에 얻는다.

상수로 쓰지 않는다

4096을 박아 두면 16 KiB 페이지인 macOS에서 틀린다.

pagetouch.c · 66–77행 · 크기 계산과 malloc

```
66      size_t size = (size_t)mib * 1024 * 1024;
67      size_t pages = size / (size_t)page;
68      printf("pid=%ld size_mib=%ld page=%ld pages=%zu\n",
69             (long)getpid(), mib, page, pages);
70      report("start");
71      /* volatile: 해제 직전의 쓰기를 컴파일러가 지우지 못하게 한다 */
72      volatile char *buf = malloc(size);
73      if (buf == NULL) {
74          perror("malloc");
75          return 1;
76      }
77      report("after_malloc");
```

labs/class07-02/pagetouch.c · 총 94행

66–67행 · 계산

바이트 수와 페이지 수를 size_t로 계산한다.

68–70행 · 시작

PID와 크기를 알리고 첫 측정을 한다.

71–77행 · malloc

volatile 포인터로 받고 NULL을 검사한다.

pagetouch.c · 78–94행 · 건드리기와 해제

```
78      for (size_t i = 0; i < pages / 2; i++)
79          buf[i * (size_t)page] = 1;
80      report("touch_half");
81      for (size_t i = pages / 2; i < pages; i++)
82          buf[i * (size_t)page] = 1;
83      report("touch_all");
84      for (size_t i = 0; i < pages; i++)
85          buf[i * (size_t)page] = 2;
86      report("touch_again");
87      free((void *)buf);
88      report("after_free");
89      if (fflush(stdout) == EOF) {
90          perror("stdout");
91          return 1;
92      }
93      return 0;
94  }
```

labs/class07-02/pagetouch.c · 총 94행

78–83행 · 처음 쓰기

절반, 나머지 절반을
페이지마다 한 바이트씩.

84–86행 · 두 번째

이미 연결된 페이지라
폴트가 늘지 않아야 한다.

87–88행 · free

큰 블록은 munmap되어
RSS가 돌아온다(glibc).

pstat.c · 1–21행 · 헤더, 주석, 결과 구조체

```
001  #include <stdio.h>
002  #include <stdlib.h>
003  #include <string.h>
004  #include <unistd.h>
005
006  /* /proc/PID/stat 한 줄을 해석해 미니 top 한 줄을 만든다(Linux 전용).
007     두 번째 필드 comm은 괄호로 감싼 실행 파일 이름인데 공백이나 ')'가
008     들어갈 수 있으므로, 마지막 ')'를 strrchr로 찾은 뒤부터 나눈다.
009     utime·stime은 클럭 틱 단위이므로 sysconf(_SC_CLK_TCK)로 나눈다.
010     [samples]번 1초 간격으로 읽어 두 표본의 CPU 틱 차이로 %CPU를 낸다. */
011
012  struct pstat {
013      char comm[64];
014      char state;
015      long ppid;
016      unsigned long minflt, majflt, utime, stime;
017      long threads;
018      unsigned long vsize;
019      long rss_pages;
020  };
021
```

labs/class07-02/pstat.c · 총 117행

6–8행 · comm 경고

이름에 공백과)가 있을 수 있어 마지막)를 찾는다.

9–10행 · 단위

시간은 틱, %CPU는 두 표본의 차이로 계산한다.

12–20행 · struct pstat

필요한 아홉 필드와 이름만 담는다.

pstat.c · 22–41행 · read_stat: 읽고 이름 자르기

```
022  static int read_stat(long pid, struct pstat *ps)
023  {
024      char path[64], line[1024];
025      snprintf(path, sizeof path, "/proc/%ld/stat", pid);
026      FILE *fp = fopen(path, "r");
027      if (fp == NULL)
028          return -1;
029      char *ok = fgets(line, sizeof line, fp);
030      fclose(fp);
031      if (ok == NULL)
032          return -1;
033      char *open = strchr(line, '(');
034      char *close = strrchr(line, ')');
035      if (open == NULL || close == NULL || close < open)
036          return -1;
037      size_t n = (size_t)(close - open - 1);
038      if (n >= sizeof ps->comm)
039          n = sizeof ps->comm - 1;
040      memcpy(ps->comm, open + 1, n);
041      ps->comm[n] = '\0';
```

labs/class07-02/pstat.c · 총 117행

24–32행 · 한 줄 읽기

/proc/PID/stat를 열어 한 줄을 읽고 바로 닫는다.

33–36행 · 괄호 찾기

첫 (와 마지막)를 찾고 순서를 검증한다.

37–41행 · 이름 복사

버퍼 크기로 잘라 복사하고 \0으로 끝낸다.

pstat.c · 42–57행 · sscanf로 필드 읽기

```
042      /* 필드 3(state)부터: %*를 붙인 필드는 읽고 버린다 */
043      int got = sscanf(close + 2,
044          "%c %ld %*d %*d %*d %*d %*u %lu %*u %lu %*u %lu %lu "
045          "%*d %*d %*d %*d %ld %*d %*u %lu %ld",
046          &ps->state, &ps->ppid, &ps->minflt, &ps->majflt,
047          &ps->utime, &ps->stime, &ps->threads, &ps->vsize,
048          &ps->rss_pages);
049      return got == 9 ? 0 : -1;
050  }
051
052  int main(int argc, char **argv)
053  {
054      if (argc < 2 || argc > 3) {
055          fprintf(stderr, "usage: %s PID|self [samples 1-60]\n", argv[0]);
056          return 2;
057      }
```

labs/class07-02/pstat.c · 총 117행

43–48행 · 서식

%*는 읽고 버린다.
필드 번호를 정확히 맞춘다.

49행 · 개수 검사

아홉 개를 다 읽어야
성공이다.

52–57행 · main 시작

PID 인자는 필수,
표본 수는 선택이다.

pstat.c · 58–76행 · PID와 표본 수 해석

```
058     long pid;
059     char *end = NULL;
060     if (strcmp(argv[1], "self") == 0) {
061         pid = (long)getpid();
062     } else {
063         pid = strtol(argv[1], &end, 10);
064         if (end == argv[1] || *end != '\0' || pid < 1) {
065             fprintf(stderr, "usage: %s PID|self [samples 1-60]\n", argv[0]);
066             return 2;
067         }
068     }
069     long samples = 1;
070     if (argc == 3) {
071         samples = strtol(argv[2], &end, 10);
072         if (end == argv[2] || *end != '\0' || samples < 1 || samples > 60) {
073             fprintf(stderr, "usage: %s PID|self [samples 1-60]\n", argv[0]);
074             return 2;
075         }
076     }
```

labs/class07-02/pstat.c · 총 117행

60–61행 · self

self면 자기 PID를 쓴다.
테스트와 시연에 편하다.

62–68행 · PID

양의 정수만 받는다.

69–76행 · 표본

1–60회. 1이면 %CPU를
계산하지 않는다.

pstat.c · 77–97행 · 첫 표본과 반복 시작

```
077      long tick = sysconf(_SC_CLK_TCK);
078      long page = sysconf(_SC_PAGESIZE);
079      struct pstat prev, cur;
080      if (read_stat(pid, &prev) == -1) {
081  #ifdef __linux__
082          fprintf(stderr, "/proc/%ld/stat: cannot read or parse\n", pid);
083          return 1;
084  #else
085          printf("stat=unsupported (no /proc on this system)\n");
086          return 0;
087  #endif
088      }
089      printf("pid=%ld comm=(%s) ppid=%ld threads=%ld clk_tck=%ld\n",
090             pid, prev.comm, prev.ppid, prev.threads, tick);
091      for (long i = 0; i < samples; i++) {
092          if (samples > 1)
093              sleep(1);
094          if (read_stat(pid, &cur) == -1) {
095              printf("gone: pid %ld exited\n", pid);
096              break;
097          }
```

labs/class07-02/pstat.c · 총 117행

77–78행 · 단위

틱과 페이지 크기를
실행 중에 얻는다.

80–88행 · 첫 읽기

Linux에서 실패는 오류,
/proc가 없으면 unsupported.

91–97행 · 반복

1초 쉬고 다시 읽는다.
사라졌으면 gone을 출력.

pstat.c · 98–117행 · %CPU 계산과 출력

```
098         unsigned long d = (cur.utime + cur.stime) - (prev.utime + prev.stime);
099         /* 표본이 하나면 비교할 구간이 없으므로 %CPU는 '-'로 둔다 */
100         if (samples > 1)
101             printf("state=%c cpu=%3.0f%% ", cur.state,
102                    100.0 * (double)d / (double)tick);
103         else
104             printf("state=%c cpu=-    ", cur.state);
105         printf("utime=%.2f stime=%.2f vsz_kib=%lu rss_kib=%ld "
106                "minflt=%lu majflt=%lu\n",
107                (double)cur.utime / tick, (double)cur.stime / tick,
108                cur.vsize / 1024, cur.rss_pages * page / 1024,
109                cur.minflt, cur.majflt);
110         prev = cur;
111     }
112     if (fflush(stdout) == EOF) {
113         perror("stdout");
114         return 1;
115     }
116     return 0;
117 }
```

labs/class07-02/pstat.c · 총 117행

98행 · 틱 차이

(utime+stime) 두 표본의 차이가 그 1초 동안의 사용량.

100–104행 · %CPU

100 × Δ / 틱. 표본이 하나면 -로 둔다.

105–110행 · 단위 변환

틱 → 초, 바이트 → KiB, 페이지 수 → KiB.

cpu_wait.c · 1–13행 · 헤더와 파일 주석

```
01  #include <fcntl.h>
02  #include <stdio.h>
03  #include <stdlib.h>
04  #include <string.h>
05  #include <sys/resource.h>
06  #include <time.h>
07  #include <unistd.h>
08
09  /* 같은 1초라도 CPU를 어떻게 쓰는지에 따라 사용량 숫자가 달라진다.
10     busy는 사용자 모드 계산, sys는 짧은 시스템 호출 반복, sleep은
11     잠자기를 반복한다. 모드마다 getrusage 차이로 user·sys CPU 시간과
12     자발적(nvcsw)·비자발적(nivcsw) 문맥 교환 횟수를 보고한다. */
13
```

labs/class07-02/cpu_wait.c · 총 90행

1–7행 · 헤더

fcntl(open), resource (getrusage), time(clock).

9–11행 · 세 모드

busy 계산, sys 짧은 호출, sleep 잠자기.

11–12행 · 측정값

user · sys 시간과 자발적 · 비자발적 교환 횟수.

cpu_wait.c · 14–28행 · 시간 보조 함수

```
14  static double now_sec(void)
15  {
16      struct timespec ts;
17      if (clock_gettime(CLOCK_MONOTONIC, &ts) == -1) {
18          perror("clock_gettime");
19          exit(1);
20      }
21      return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
22  }
23
24  static double tv(struct timeval t)
25  {
26      return (double)t.tv_sec + (double)t.tv_usec / 1e6;
27  }
28
```

labs/class07-02/cpu_wait.c · 총 90행

14–22행 · now_sec

단조 시계를 초 단위 double로 돌려준다.

24–27행 · tv

timeval(초 + 마이크로초)을 초 단위 double로 바꾼다.

두 시계

벽시계 경과와 CPU 시간을 같은 단위(초)로 맞춘다.

cpu_wait.c · 29–50행 · run: 모드별 1초

```
29  static void run(const char *mode, double seconds, int devnull)
30  {
31      struct rusage a, b;
32      if (getrusage(RUSAGE_SELF, &a) == -1) {
33          perror("getrusage");
34          exit(1);
35      }
36      double start = now_sec(), stop = start + seconds;
37      volatile unsigned long spin = 0;
38      char byte = 'x';
39      while (now_sec() < stop) {
40          if (strcmp(mode, "busy") == 0) {
41              for (int i = 0; i < 100000; i++)
42                  spin++;
43          } else if (strcmp(mode, "sys") == 0) {
44              if (write(devnull, &byte, 1) == -1) {
45                  perror("write");
46                  exit(1);
47              }
48          } else {
49              usleep(10000);   /* 10ms 잠들기 */
50          }
```

labs/class07-02/cpu_wait.c · 총 90행

31–36행 · 시작 측정

getrusage로 출발점을
기록하고 마감 시각을 정한다.

40–42행 · busy

시스템 호출 없이
덧셈만 반복한다.

43–49행 · sys · sleep

1바이트 write 반복,
또는 10ms씩 잠든다.

cpu_wait.c · 51–66행 · 차이 계산과 출력

```
51      }
52      double wall = now_sec() - start;
53      if (getrusage(RUSAGE_SELF, &b) == -1) {
54          perror("getrusage");
55          exit(1);
56      }
57      printf("mode=%-5s wall=%.2f user=%.2f sys=%.2f nvcsw=%ld nivcsw=%ld\n",
58             mode, wall, tv(b.ru_utime) - tv(a.ru_utime),
59             tv(b.ru_stime) - tv(a.ru_stime),
60             b.ru_nvcsw - a.ru_nvcsw, b.ru_nivcsw - a.ru_nivcsw);
61      if (fflush(stdout) == EOF) {
62          perror("stdout");
63          exit(1);
64      }
65  }
66
```

labs/class07-02/cpu_wait.c · 총 90행

52–56행 · 끝 측정

벽시계 경과와 두 번째 getrusage를 얻는다.

57–60행 · 차이

user · sys · nvcsw · nivcsw를 모두 차이로 출력한다.

61–64행 · flush

모드마다 바로 내보낸다.

cpu_wait.c · 67–81행 · 인자 검사와 /dev/null

```
67  int main(int argc, char **argv)
68  {
69      const char *modes[] = {"busy", "sys", "sleep"};
70      const char *only = argc == 2 ? argv[1] : NULL;
71      if (argc > 2 || (only != NULL && strcmp(only, "busy") != 0 &&
72                       strcmp(only, "sys") != 0 && strcmp(only, "sleep") != 0)) {
73          fprintf(stderr, "usage: %s [busy|sys|sleep]\n", argv[0]);
74          return 2;
75      }
76      int devnull = open("/dev/null", O_WRONLY);
77      if (devnull == -1) {
78          perror("/dev/null");
79          return 1;
80      }
81      printf("pid=%ld seconds_per_mode=1\n", (long)getpid());
```

labs/class07-02/cpu_wait.c · 총 90행

69–70행 · 모드 목록

인자가 있으면 그 모드만,
없으면 셋 다.

71–75행 · 검사

알 수 없는 모드는
사용법 오류 2.

76–81행 · /dev/null

sys 모드용 FD를
한 번만 열어 둔다.

cpu_wait.c · 82–90행 · 실행과 정리

```
82      for (int i = 0; i < 3; i++)
83          if (only == NULL || strcmp(only, modes[i]) == 0)
84              run(modes[i], 1.0, devnull);
85      if (close(devnull) == -1) {
86          perror("close");
87          return 1;
88      }
89      return 0;
90  }
```

82–84행 · 반복

선택된 모드만 run을 호출한다.

85–88행 · close

열었던 FD를 닫고 실패를 검사한다.

89행 · 종료

정상 종료 0.

labs/class07-02/cpu_wait.c · 총 90행

reap_show.c · 1–22행 · 헤더, 주석, flush 보조

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  #include <string.h>
04  #include <sys/wait.h>
05  #include <unistd.h>
06
07  /* 종료와 회수 사이의 상태를 일부러 늘려 관찰한다.
08     zombie: 자식은 곧바로 종료 상태 7로 끝나고, 부모는 [seconds]초
09             동안 회수하지 않는다. 그동안 ps에 Z <defunct>가 보인다.
10             그 뒤 waitpid로 회수하면 항목이 사라진다.
11     orphan: 부모가 먼저 끝나고, 자식은 2초 뒤 자기 PPID를 다시 본다.
12             새 부모는 PID 1이거나 가장 가까운 subreaper다.
13     fork·waitpid의 정식 계약은 10주차에서 다룬다. 여기서는 관찰 도구다. */
14
15  static int flush_or_die(void)
16  {
17      if (fflush(stdout) == EOF) {
18          perror("stdout");
19          return -1;
20      }
21      return 0;
22  }
```

labs/class07-02/reap_show.c · 총 92행

4행 · sys/wait.h

waitpid와 WIFEXITED · WEXITSTATUS 매크로.

7–13행 · 두 모드

zombie는 회수를 미루고, orphan은 부모가 먼저 끝난다.

15–22행 · flush_or_die

fork 전후 출력이 섞이지 않게 매번 비운다.

reap_show.c · 24–40행 · zombie: 회수 미루기

```
24  static int zombie(long seconds)
25  {
26      pid_t child = fork();
27      if (child == -1) {
28          perror("fork");
29          return 1;
30      }
31      if (child == 0)
32          _exit(7);               /* 자식: 즉시 종료 상태 7 */
33      printf("parent=%ld child=%ld (child exited, not reaped yet)\n",
34             (long)getpid(), (long)child);
35      printf("observe: ps -o pid,ppid,stat,command -p %ld\n", (long)child);
36      if (flush_or_die() == -1)
37          return 1;
38      unsigned int left = (unsigned int)seconds;
39      while (left != 0)
40          left = sleep(left);
```

labs/class07-02/reap_show.c · 총 92행

26–30행 · fork

실패하면 -1, 성공하면
부모와 자식이 둘 다 돌아온다.

31–32행 · 자식

반환값 0인 쪽이 자식이다.
곧바로 _exit(7).

33–40행 · 부모

PID를 알리고 seconds초
동안 회수 없이 잔다.

reap_show.c · 41–51행 · waitpid로 회수

```
41      int status;
42      if (waitpid(child, &status, 0) == -1) {
43          perror("waitpid");
44          return 1;
45      }
46      if (WIFEXITED(status))
47          printf("reaped child=%ld exit_status=%d\n",
48                 (long)child, WEXITSTATUS(status));
49      return flush_or_die() == -1 ? 1 : 0;
50  }
51
```

labs/class07-02/reap_show.c · 총 92행

41–45행 · waitpid

특정 자식 PID를 기다려
종료 상태를 받는다.

46–48행 · 해석

정상 종료면 WEXITSTATUS로
종료 코드 7을 꺼낸다.

49행 · 반환

flush 실패면 1,
아니면 0.

reap_show.c · 52–73행 · orphan: 부모가 먼저

```
52  static int orphan(void)
53  {
54      if (flush_or_die() == -1)
55          return 1;
56      pid_t child = fork();
57      if (child == -1) {
58          perror("fork");
59          return 1;
60      }
61      if (child > 0) {
62          printf("parent=%ld exits now, child=%ld\n",
63                 (long)getpid(), (long)child);
64          return flush_or_die() == -1 ? 1 : 0;
65      }
66      printf("child=%ld ppid_before=%ld\n", (long)getpid(), (long)getppid());
67      if (flush_or_die() == -1)
68          _exit(1);
69      sleep(2);
70      printf("child=%ld ppid_after=%ld (adopted)\n",
71             (long)getpid(), (long)getppid());
72      _exit(flush_or_die() == -1 ? 1 : 0);
73  }
```

labs/class07-02/reap_show.c · 총 92행

54–60행 · fork 전 flush

버퍼를 비우고 fork한다.

61–65행 · 부모

자식 PID를 알리고
곧바로 끝난다.

66–72행 · 자식

PPID를 보고, 2초 잔 뒤
다시 PPID를 본다.

reap_show.c · 74–92행 · main: 모드 선택

```
74
75  int main(int argc, char **argv)
76  {
77      if (argc >= 2 && argc <= 3 && strcmp(argv[1], "zombie") == 0) {
78          long seconds = 10;
79          if (argc == 3) {
80              char *end = NULL;
81              seconds = strtol(argv[2], &end, 10);
82              if (end == argv[2] || *end != '\0' || seconds < 0 || seconds > 600)
83                  seconds = -1;
84          }
85          if (seconds >= 0)
86              return zombie(seconds);
87      } else if (argc == 2 && strcmp(argv[1], "orphan") == 0) {
88          return orphan();
89      }
90      fprintf(stderr, "usage: %s zombie [seconds 0-600] | orphan\n", argv[0]);
91      return 2;
92  }
```

labs/class07-02/reap_show.c · 총 92행

77–86행 · zombie

초 인자는 0–600.
잘못되면 -1로 표시한다.

87–88행 · orphan

인자가 정확히 하나일 때만.

90–91행 · 사용법

어느 쪽도 아니면
종료 상태 2.

limit_lab.c · 1–17행 · 헤더와 파일 주석

```
001  #include <errno.h>
002  #include <fcntl.h>
003  #include <stdio.h>
004  #include <stdlib.h>
005  #include <string.h>
006  #include <sys/resource.h>
007  #include <unistd.h>
008
009  /* 자원 한계를 낮춘 뒤 일부러 넘겨서 "넘으면 무엇이 일어나는가"를 본다.
010     nofile N : soft NOFILE을 N으로 낮추고 /dev/null을 실패할 때까지 연다.
011                EMFILE로 실패하며 열린 개수는 N - 3(표준 스트림)이다.
012     as MiB   : soft AS를 MiB로 낮추고 그 두 배를 malloc한다. NULL과
013                ENOMEM이 돌아온다(Linux. 다른 시스템은 결과가 다를 수 있다).
014     cpu S    : soft CPU를 S초로 낮추고 계속 계산한다. 한계를 넘으면
015                커널이 SIGXCPU를 보내 기본 동작으로 종료된다.
016     한계는 이 프로세스(와 이후의 자식)에만 적용되고 셸에는 영향이 없다. */
017
```

labs/class07-02/limit_lab.c · 총 108행

9–11행 · nofile

soft NOFILE을 낮추고 EMFILE까지 연다.

12–13행 · as

soft AS를 낮추고 그 두 배를 malloc한다.

14–16행 · cpu와 범위

SIGXCPU로 끝난다. 한계는 이 프로세스와 자식에만.

limit_lab.c · 18–35행 · lower_soft: 읽고 고치고 쓰기

```
018  /* soft 한계를 value로 낮추고 실제로 적용된 값을 돌려준다(실패 시 -1) */
019  static long long lower_soft(int resource, rlim_t value)
020  {
021      struct rlimit lim;
022      if (getrlimit(resource, &lim) == -1) {
023          perror("getrlimit");
024          return -1;
025      }
026      if (lim.rlim_max != RLIM_INFINITY && value > lim.rlim_max)
027          value = lim.rlim_max;   /* soft는 hard를 넘을 수 없다 */
028      lim.rlim_cur = value;
029      if (setrlimit(resource, &lim) == -1) {
030          perror("setrlimit");
031          return -1;
032      }
033      return (long long)value;
034  }
035
```

labs/class07-02/limit_lab.c · 총 108행

21–25행 · 읽기

현재 soft · hard 쌍을
먼저 읽는다.

26–28행 · 자르기

hard보다 크면 hard로
낮춰 soft에 넣는다.

29–33행 · 쓰기

setrlimit 후 실제로
적용한 값을 돌려준다.

limit_lab.c · 36–57행 · nofile: EMFILE까지 열기

```
036  static int nofile(long n)
037  {
038      long long soft = lower_soft(RLIMIT_NOFILE, (rlim_t)n);
039      if (soft == -1)
040          return 1;
041      int fds[1024];
042      int count = 0;
043      while (count < 1024) {
044          int fd = open("/dev/null", O_RDONLY);
045          if (fd == -1) {
046              printf("open failed after %d opens: errno=%d (%s)\n",
047                     count, errno, strerror(errno));
048              break;
049          }
050          fds[count++] = fd;
051      }
052      printf("last_fd=%d soft_limit=%lld\n", count ? fds[count - 1] : -1, soft);
053      for (int i = 0; i < count; i++)
054          close(fds[i]);
055      printf("closed=%d\n", count);
056      return 0;
057  }
```

labs/class07-02/limit_lab.c · 총 108행

38–40행 · 낮추기

실제 적용된 soft를 받아 둔다.

43–51행 · 열기 반복

실패하면 errno와 문자열을 출력하고 멈춘다.

52–55행 · 정리

마지막 FD와 한계를 알리고 연 FD를 모두 닫는다.

limit_lab.c · 58–75행 · as: 주소 공간 한계

```
058
059  static int as_limit(long mib)
060  {
061      if (lower_soft(RLIMIT_AS, (rlim_t)mib * 1024 * 1024) == -1)
062          return 1;
063      size_t want = (size_t)mib * 2 * 1024 * 1024;
064      errno = 0;
065      void *p = malloc(want);
066      if (p == NULL) {
067          printf("malloc(%ld MiB) = NULL errno=%d (%s)\n",
068                 mib * 2, errno, strerror(errno));
069      } else {
070          printf("malloc(%ld MiB) succeeded (limit not enforced here)\n",
071                 mib * 2);
072          free(p);
073      }
074      return 0;
075  }
```

labs/class07-02/limit_lab.c · 총 108행

61–62행 · 낮추기

MiB를 바이트로 바꿔
soft AS를 낮춘다.

63–65행 · 두 배 요청

errno를 0으로 두고
한계의 두 배를 malloc한다.

66–73행 · 결과

NULL이면 errno를,
성공하면 강제되지 않았다고.

limit_lab.c · 76–91행 · cpu: CPU 시간 한계

```
076
077  static int cpu_limit(long seconds)
078  {
079      if (lower_soft(RLIMIT_CPU, (rlim_t)seconds) == -1)
080          return 1;
081      printf("pid=%ld spinning past %ld s of CPU...\n",
082             (long)getpid(), seconds);
083      if (fflush(stdout) == EOF) {
084          perror("stdout");
085          return 1;
086      }
087      volatile unsigned long spin = 0;
088      for (;;)
089          spin++;
090  }
091
```

labs/class07-02/limit_lab.c · 총 108행

79–80행 · 낮추기

soft CPU를 초 단위로 낮춘다.

81–86행 · 알림

PID를 알리고 바로 내보낸다.

87–89행 · 무한 루프

반환하지 않는다. 커널이 SIGXCPU로 끝낸다.

limit_lab.c · 92–108행 · main: 모드와 범위

```
092  int main(int argc, char **argv)
093  {
094      if (argc == 3) {
095          char *end = NULL;
096          long v = strtol(argv[2], &end, 10);
097          int valid = end != argv[2] && *end == '\0';
098          if (valid && strcmp(argv[1], "nofile") == 0 && v >= 4 && v <= 1024)
099              return nofile(v);
100          if (valid && strcmp(argv[1], "as") == 0 && v >= 16 && v <= 4096)
101              return as_limit(v);
102          if (valid && strcmp(argv[1], "cpu") == 0 && v >= 1 && v <= 10)
103              return cpu_limit(v);
104      }
105      fprintf(stderr, "usage: %s nofile N(4-1024) | as MiB(16-4096)"
106              " | cpu S(1-10)\n", argv[0]);
107      return 2;
108  }
```

labs/class07-02/limit_lab.c · 총 108행

94–97행 · 숫자 검사

두 번째 인자가 정수인지 한 번만 검사한다.

98–103행 · 범위

모드마다 안전한 범위를 정했다.

105–107행 · 사용법

어느 것도 맞지 않으면 종료 상태 2.